Differentiable Lattice Boltzmann Simulation with Second-Order Boundary Conditions for Gradient-Based Shape Optimisation

Marcos Ashton Iglesias

Department of Computer Science, University of Exeter, Exeter, UK

Abstract

We present a fully differentiable D3Q19 Lattice Boltzmann solver in JAX that supports gradient-based shape optimisation through second-order accurate Bouzidi interpolated bounce-back boundary conditions. The Bouzidi fractional wall distances are computed via differentiable ray-surface intersection, allowing gradients of aerodynamic quantities to flow through the boundary treatment into obstacle geometry parameters. This is in contrast to prior work that either uses first-order standard bounce-back (which provides no useful shape gradient) or treats wall distances as fixed. We validate the solver against the analytic Poiseuille flow solution and experimental sphere drag data at $Re = 100$ (Clift et al. 1978). We demonstrate gradient-based drag minimisation on a sphere, achieving 68% drag reduction, and show that Bouzidi boundary gradients are approximately 35× stronger than standard bounce-back gradients, producing a 37% better optimum. The solver, including BGK and MRT collision operators, is open-source at `https://github.com/MarcosAsh/LBM_JAX_Autodiff`.

Keywords: lattice Boltzmann method, automatic differentiation, shape optimisation, Bouzidi bounce-back, JAX, inverse design

1 Introduction

The Lattice Boltzmann Method (LBM) has become a standard tool for computational fluid dynamics, particularly for flows around complex geometries where its regular grid structure and local update rules give it an advantage over unstructured mesh methods [1]. A natural next step is to make LBM simulations differentiable, so that gradients of flow quantities with respect to design parameters can be computed efficiently. These gradients enable gradient-based optimisation of boundary conditions, material properties, and obstacle geometry without resorting to finite-difference sweeps or evolutionary methods.

Adjoint-based sensitivity analysis for LBM has been explored by several groups. Tekitek et al. [2] derived adjoint equations for LBM and applied them to parameter identification. Krause et al. [3] extended this to shape optimisation within the OpenLB framework. More recently, automatic differentiation (AD) through fluid simulations has gained traction through tools like PhiFlow [4], which provides a differentiable Navier-Stokes solver, and through JAX-based implementations that exploit reverse-mode AD without requiring manual adjoint derivation.

A critical gap in the existing literature concerns boundary treatment. Most differentiable LBM implementations use standard halfway bounce-back, which places the effective wall exactly at the midpoint between fluid and solid nodes. This is first-order accurate and, more importantly for optimisation, provides no useful gradient with respect to wall position: the wall location is locked to the grid, so the gradient of any flow quantity with respect to geometry is zero almost everywhere (it changes only when a cell flips between solid and fluid).

Bouzidi interpolated bounce-back [5] resolves this by using the fractional distance $q \in [0, 1]$ from the fluid node to the wall along each lattice link. The streaming step interpolates the bounced distribution using q, giving second-order wall accuracy on curved surfaces. The key observation is that q is a continuous, differentiable function of the obstacle geometry. For a sphere of radius r, the ray-sphere intersection formula gives q as a smooth function of r. Reverse-mode AD traces through this formula automatically, delivering $\partial C_d/\partial r$ through the chain

$$\frac{\partial C_d}{\partial r} = \sum_{\text{links}} \frac{\partial C_d}{\partial f_i} \cdot \frac{\partial f_i}{\partial q} \cdot \frac{\partial q}{\partial r}. \tag{1}$$

In this paper we present a D3Q19 LBM solver implemented entirely in JAX [12] that makes this gradient chain concrete. Every component of the solver, from collision to streaming to force computation, is a pure function compatible with `jax.grad`. We validate the solver against analytic solutions and experimental data, and demonstrate gradient-based shape optimisation with

Bouzidi boundary conditions. We compare the optimisation performance of Bouzidi gradients against standard bounce-back gradients on the same problem, providing direct evidence that second-order boundary treatment improves gradient quality for shape optimisation.

2 Methodology

2.1 D3Q19 lattice Boltzmann

We use the three-dimensional D3Q19 lattice with 19 discrete velocity directions: one rest, six axis-aligned, and twelve diagonal. The quadrature weights are $w_0 = 1/3$, $w_{1\text{–}6} = 1/18$, and $w_{7\text{–}18} = 1/36$. The lattice speed of sound squared is $c_s^2 = 1/3$.

The equilibrium distribution at each node is

$$f_i^{\text{eq}} = w_i \rho \left(1 + \frac{\mathbf{e}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{e}_i \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u} \cdot \mathbf{u}}{2c_s^2}\right), \quad (2)$$

where $\rho = \sum_i f_i$ is the macroscopic density and $\rho\mathbf{u} = \sum_i f_i \mathbf{e}_i$ is the momentum.

2.2 Collision operators

2.2.1 BGK (single relaxation time)

The Bhatnagar-Gross-Krook operator [6] relaxes each distribution toward equilibrium at a uniform rate:

$$f_i^{\text{out}} = f_i - \frac{1}{\tau}(f_i - f_i^{\text{eq}}), \quad (3)$$

where τ is the relaxation time. The kinematic viscosity is $\nu = c_s^2(\tau - 1/2) = (\tau - 1/2)/3$.

2.2.2 MRT (multiple relaxation time)

The MRT operator [7] transforms to moment space via a matrix M, relaxes each moment independently, and transforms back:

$$\mathbf{f}^{\text{out}} = \mathbf{f} - M^{-1} S (M\mathbf{f} - M\mathbf{f}^{\text{eq}}), \quad (4)$$

where $S = \text{diag}(s_0, \ldots, s_{18})$ contains the per-moment relaxation rates. Conserved moments (density, momentum) have $s_k = 0$. Stress moments relax at the physical rate $s_k = 1/\tau$. Ghost and higher-order moments are damped aggressively ($s_k = 1.2$–1.98) following [7]. Our implementation uses the sparse forward and inverse transforms, avoiding a full 19×19 matrix multiply.

2.3 Bouzidi interpolated bounce-back

At solid boundaries, distributions that would stream from inside the wall are replaced by bounced values. The Bouzidi scheme [5] uses the fractional distance q from the fluid node to the wall along the lattice link:

$$f_i = \begin{cases} \frac{1}{2q} f_{i'}^* + \left(1 - \frac{1}{2q}\right) f_i^* & q \geq 0.5, \\ 2q\, f_{i'}^* + (1 - 2q)\, f_{i'}^*(\mathbf{x} + \mathbf{e}_i) & q < 0.5, \end{cases} \quad (5)$$

where i' is the opposite direction and f^* denotes post-collision values.

For a sphere of radius r centred at $\mathbf{c}$, the wall distance along a lattice link from fluid node $\mathbf{x}$ in direction $\mathbf{e}_i$ is obtained by solving the ray-sphere intersection:

$$q = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \quad (6)$$

with $a = |\mathbf{d}|^2$, $b = 2(\mathbf{x} - \mathbf{c}) \cdot \mathbf{d}$, $c = |\mathbf{x} - \mathbf{c}|^2 - r^2$, where $\mathbf{d}$ is the lattice velocity vector in world coordinates. This is a smooth function of r, and JAX's reverse-mode AD traces through it automatically.

2.4 Differentiable solid representation

The binary solid mask is a step function with zero gradient almost everywhere. Following List et al. [11], we replace it with a sigmoid-smoothed porosity field:

$$\phi(\mathbf{x}) = \sigma(\alpha(r - |\mathbf{x} - \mathbf{c}|)), \quad (7)$$

where σ is the logistic sigmoid and α controls the transition sharpness. The collision step blends between fluid and solid behaviour:

$$f_i^{\text{out}} = (1 - \phi) f_i^{\text{collided}} + \phi\, f_{i'}^{\text{collided}}. \quad (8)$$

This provides a differentiable path through collision, complementing the Bouzidi gradient path through streaming.

2.5 Force computation

We use the Mei-Luo-Shyy momentum exchange method [9]. For each fluid node adjacent to a solid boundary, the force contribution from lattice link i is $\mathbf{F}_i = \mathbf{e}_i(f_i^* + f_{i'})$. The drag coefficient is

$$C_d = \frac{|F_x|}{\frac{1}{2}\rho_\infty U_\infty^2 A}, \quad (9)$$

where A is the projected frontal area.

2.6 Boundary conditions

At the inlet ($x = 0$) we apply Zou-He velocity boundary conditions [10]. At the outlet ($x = N_x - 1$) we apply Zou-He with prescribed pressure ($\rho = 1$). The y and z boundaries are periodic.

2.7 Gradient checkpointing

The simulation is compiled into a single XLA program via `jax.lax.scan`. Each step is wrapped with `jax.checkpoint`, which recomputes forward values during the backward pass instead of storing them. Memory drops from $O(N)$ intermediate states to $O(1)$.

Table 1: Poiseuille flow: LBM vs analytic velocity. $H = 18$, $\tau = 0.8$, 3000 steps.

y	u_x (LBM)	u_x (analytic)
0	0.0002	0.0000
1	0.0082	0.0083
5	0.0579	0.0577
9	0.0769	0.0766
10	0.0769	0.0766
15	0.0484	0.0482
19	0.0002	0.0000

3 Validation

3.1 Poiseuille flow

We simulate pressure-driven flow between two parallel walls separated by $H = 18$ lattice cells in a $60 \times 20 \times 4$ domain. The inlet velocity is $U = 0.05$, the relaxation time is $\tau = 0.8$ ($\nu = 0.1$), and we run 3000 BGK steps.

Table 1 shows the comparison against the analytic parabolic solution. The LBM solution matches to three or more significant figures across the channel, with perfect top-bottom symmetry.

3.2 Sphere drag at $Re = 100$

A sphere of diameter $D = 16$ lattice cells is placed in a $128 \times 64 \times 64$ domain with $U = 0.05$, giving $Re = 100$ and $\tau = 0.524$. We run 4000 MRT steps.

The Clift et al. [8] reference value is $C_d = 1.09$. Our MRT simulation produces $C_d = 1.20$, a relative error of 10.3%. This is consistent with geometric discretisation error at $D = 16$ cells. The BGK operator on the same grid produces $C_d = 0.75$ with large oscillations and rising velocity magnitude, confirming the advantage of MRT for stability near the lower limit of τ.

3.3 Gradient verification

We verify $\partial C_d / \partial r$ against central finite differences with step size $\epsilon = 10^{-3}$. On a $32 \times 16 \times 16$ grid with the differentiable sphere geometry, `jax.grad` returns a finite, nonzero gradient that agrees in sign with the finite-difference estimate. The tests pass on both CPU and NVIDIA A100 GPU.

4 Shape optimisation

4.1 Problem setup

We demonstrate gradient-based drag minimisation on a sphere. The design variable is the sphere radius r. The objective is $\min_r C_d(r)$, computed by a differentiable forward simulation on a $48 \times 24 \times 24$ grid at $Re = 50$. The sigmoid-smoothed solid representation and differentiable Bouzidi q values provide the gradient signal.

Table 2: Bouzidi vs standard bounce-back: same grid, physics, and optimiser. Only the wall treatment differs.

	Bouzidi (q from intersection)	Standard ($q = 0.5$)
Best C_d achieved	0.405	0.645
Avg $\lvert\partial C_d/\partial r\rvert$	7.74	0.22
C_d reduction vs initial	37.3%	5.9%

4.2 Optimisation results

Starting from $r = 0.5$ ($C_d = 0.374$), gradient descent reduces the drag coefficient to $C_d = 0.119$ over 25 iterations, a 68% reduction. The optimised radius is $r = 0.59$. This demonstrates that meaningful gradients flow through the entire pipeline: geometry parameters $\rightarrow$ sigmoid porosity $\rightarrow$ soft bounce-back $\rightarrow$ Bouzidi streaming $\rightarrow$ momentum exchange $\rightarrow$ drag coefficient.

4.3 Bouzidi vs standard bounce-back

To isolate the effect of boundary treatment on gradient quality, we run the same optimisation problem twice: once with Bouzidi q values from ray-sphere intersection, and once with $q = 0.5$ everywhere (standard midpoint bounce-back). Both use the same grid ($48 \times 24 \times 24$), physics ($Re = 50$, BGK), and optimiser (Adam, $lr = 3 \times 10^{-3}$, 25 iterations, 150 steps per evaluation).

Table 2 summarises the results from an NVIDIA A100 GPU.

The Bouzidi gradients are approximately $35\times$ larger in magnitude than the standard bounce-back gradients. The Bouzidi q values carry continuous information about wall position within each lattice cell, providing a rich gradient signal. Standard bounce-back reduces this to a binary choice (wall at midpoint or not), which nearly flattens the gradient landscape. The optimisation with Bouzidi reaches a C_d that is 37% lower than the best standard bounce-back result. This is the central finding: second-order boundary treatment produces qualitatively better shape gradients.

5 Implementation

The solver is implemented in Python using JAX [12]. Every function is pure and compatible with `jax.jit`. The simulation state is a NamedTuple containing the distribution array $f \in \mathbb{R}^{19 \times N_x \times N_y \times N_z}$, the solid mask, and the Bouzidi q buffer.

Streaming uses `jnp.roll` for periodic shifting and `jnp.where` for Bouzidi interpolation. The MRT collision uses `jax.vmap` over spatial cells with sparse mo-

ment transforms. The simulation loop compiles via `jax.lax.scan` into a single XLA program.

On an NVIDIA A100, a single BGK step on a 128^3 grid takes approximately 15 ms. MRT takes approximately 40 ms. The backward pass adds roughly $2\times$ overhead with checkpointing enabled.

The code is open-source at `https://github.com/MarcosAsh/LBM_JAX_Autodiff` and includes 49 automated tests and a Modal cloud GPU worker for reproducibility.

6 Conclusion

We have presented a differentiable D3Q19 Lattice Boltzmann solver that supports gradient-based shape optimisation through Bouzidi interpolated bounce-back boundary conditions. The solver is validated against the Poiseuille analytic solution (three-digit agreement) and experimental sphere drag at $Re = 100$ (10% of the Clift reference with MRT on a 16-cell diameter grid).

The central contribution is making the Bouzidi fractional wall distances differentiable with respect to geometry parameters. We showed that this produces gradients $35\times$ stronger than standard bounce-back, and that optimisation with Bouzidi gradients reaches a 37% better drag minimum than standard bounce-back on the same problem.

Future work includes extending the differentiable boundary treatment to mesh-based geometries via differentiable voxelisation, applying the solver to more complex shapes (Ahmed body drag reduction), and performing a full grid convergence study of the gradient at higher resolutions.

Data availability

Source code, tests, and examples: `https://github.com/MarcosAsh/LBM_JAX_Autodiff`.

References

[1] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E.M. Viggen, *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2017.

[2] M.M. Tekitek, M. Bouzidi, F. Dubois, and P. Lallemand, "Adjoint lattice Boltzmann equation for parameter identification," *Computers & Fluids*, vol. 35, pp. 805–813, 2006.

[3] M.J. Krause, G. Thäter, and V. Heuveline, "Adjoint-based fluid flow control and optimisation with lattice Boltzmann methods," *Computers & Mathematics with Applications*, vol. 65, pp. 945–960, 2013.

[4] P. Holl, V. Koltun, and N. Thuerey, "Learning to Control PDEs with Differentiable Physics," *ICLR*, 2020.

[5] M. Bouzidi, M. Firdaouss, and P. Lallemand, "Momentum transfer of a Boltzmann-lattice fluid with boundaries," *Physics of Fluids*, vol. 13, no. 11, pp. 3452–3459, 2001.

[6] P.L. Bhatnagar, E.P. Gross, and M. Krook, "A Model for Collision Processes in Gases," *Physical Review*, vol. 94, no. 3, pp. 511–525, 1954.

[7] D. d'Humières, I. Ginzburg, M. Krafczyk, P. Lallemand, and L.S. Luo, "Multiple-relaxation-time lattice Boltzmann models in three dimensions," *Phil. Trans. R. Soc. A*, vol. 360, pp. 437–451, 2002.

[8] R. Clift, J.R. Grace, and M.E. Weber, *Bubbles, Drops, and Particles*, Academic Press, 1978.

[9] R. Mei, L.S. Luo, and W. Shyy, "Force evaluation in the lattice Boltzmann method involving curved geometry," *Physical Review E*, vol. 65, 041203, 2002.

[10] Q. Zou and X. He, "On pressure and velocity boundary conditions for the lattice Boltzmann BGK model," *Physics of Fluids*, vol. 9, no. 6, pp. 1591–1598, 1997.

[11] B. List, L. Chen, and N. Thuerey, "Learned Turbulence Modelling with Differentiable Fluid Solvers," *NeurIPS Workshop on Machine Learning and the Physical Sciences*, 2022.

[12] J. Bradbury et al., "JAX: composable transformations of Python+NumPy programs," 2018, `http://github.com/jax-ml/jax`.